

Project Deep Dive — LLM Orchestration + Early Ranking System

Complete Interview Script (Unified Narrative)

The script below is designed as **one continuous talk** (~4-5 minutes for initial overview). Embedded [HOOK] markers are natural pause points — if the interviewer wants to go deeper, you expand on that section. If not, you keep flowing.

Full script:

"Sure. One of the largest projects I led recently was redesigning the personalization and ranking system for our ad recommendation surfaces — both on Amazon search and cross-surface partner placements like Pinterest.

[Problem & Motivation]

When I joined the team, the system was highly fragmented. Each widget — each recommendation placement — owned its own independent sourcing and ranking pipeline. This created duplicated logic, inconsistent optimization objectives, and very slow experimentation cycles. More importantly, I did an analysis and found that 63% of search traffic came from mid-funnel queries like 'running shoes' — users who haven't committed to a specific product yet. Of those users, 35% couldn't convert into a purchase on search page 1. So there's a massive gap: these users have intent but the system wasn't smart enough to understand and act on it. That motivated the core question: can we better understand what the customer actually wants, and use that understanding to dynamically guide both sourcing and ranking?

[Architecture Overview]

As the tech lead, I designed and built a unified architecture with two core components. The first is an LLM-based orchestration layer that sits between query understanding and retrieval. The second is a placement-aware early ranking model that adapts to heterogeneous surface objectives. Let me walk through each one. [HOOK — interviewer can ask to go deeper on either]

[Component 1: LLM Orchestration]

The orchestration layer's job is: given a user's session behavior and current query, decide whether we have enough signal to personalize, and if so, generate structured child queries that guide downstream retrieval.

Concretely — the model takes the user's recent interaction sequence: views, clicks, add-to-carts, purchases, along with their current search query. It outputs a JSON-structured response with three fields: an intent classification, a confidence score between 0 and 1, and a list of 3 to 5 child queries representing the user's refined shopping intent.

For example, if a user searched 'running shoes' and I can see from their session

that they previously viewed Nike Pegasus, Brooks Ghost, and Hoka Bondi — all cushioned daily trainers — the model infers: intent is 'daily training cushioned running shoes', confidence 0.85, and generates specific child queries like 'cushioned road running shoes for daily training' and 'neutral max cushion marathon trainers.'

Then the decision logic: if confidence exceeds 0.7, we route these child queries into our semantic retrieval system — a dual encoder with ANN search — to produce personalized candidate sets. If confidence is below threshold, meaning the user's behavior is too stochastic to reliably infer intent, we fall back to a discovery-oriented strategy that expands the original query into related product categories. This ensures we maintain coverage and exploration even when personalization confidence is low. [HOOK — can dive into retrieval integration challenges]

[Evolution & Trade-offs] [HOOK — interviewer often asks about this]

The system went through two iterations driven by latency trade-offs. In V1, we used a 32B model in a semi-real-time T-1 pipeline — we processed the user's previous session offline and cached results keyed by customer ID and product type. This validated the approach: LLM-based personalization genuinely improved intent understanding. But it had two key limitations. First, the product-type cache key was too coarse — 'bedroom lamp' and 'living room lamp' mapped to the same key, causing granularity mismatches. Second, 23-second inference meant we missed users who returned quickly.

These observations motivated V2: we distilled the 32B model into a task-specific 7B model — Qwen-7B — fine-tuned on 500K teacher-generated examples. This brought inference under 100ms. We also added an intent detection gate upstream — a lightweight logistic regression classifier on session features that decides whether to invoke the LLM at all. This reduced unnecessary inference by about 60% and kept our p99 latency under 100ms. [HOOK — can discuss distillation details, prompt design, evaluation]

[Component 2: Placement-Aware Ranking]

Once we have candidates from retrieval, we need to rank them. The challenge is that different placements — the sponsored carousel on Amazon search, the sidebar, the Pinterest partner surface — have fundamentally different user behavior patterns and business objectives. We can't just train one ranker for all of them.

I designed a PEPNet-inspired architecture — this is from KDD 2023. The core idea is a shared bottom network that learns general ranking features across all placements, combined with placement-specific gating networks. Each placement has a learned embedding vector. That embedding feeds into a gating network that outputs a gate vector, which element-wise multiplies the shared representation. So the model shares statistical strength across placements — critical for cold-start surfaces with limited data — while adapting its behavior for each surface through the gate.

On top of this shared-plus-gated structure, I have task-specific towers: one for predicted CTR, one for predicted CVR, and one for relevance scoring. The final ranking score combines these with placement-specific weights: relevance,

pCTR for monetization, pCVR for advertiser ROI, and a diversity penalty to avoid advertiser concentration. Weights are tuned per-placement using counterfactual replay evaluation offline, then validated through online A/B tests. [HOOK — can discuss multi-objective trade-offs, how weights are set]

[Results & Impact]

The end-to-end system unified previously fragmented pipelines into a single ML platform with real-time personalization. In A/B testing at 15% traffic, we observed 0.18% improvement in click engagement, 0.11% improvement in downstream purchase, and 10% RoAS lift on partner surfaces — translating to \$40M incremental ad revenue within two weeks. Beyond the metrics, the unified architecture enabled other teams to plug in new placements without building their own pipelines from scratch — that scalability was actually one of the most valued outcomes.

[Leadership] [HOOK — interviewer will often ask about this separately]

In terms of team execution — this spanned intent modeling, retrieval integration, ranking, and system infrastructure. I decomposed it into clear modules and assigned ownership based on each person's strengths. I stayed hands-on for the orchestration layer since it was the highest-risk, highest-ambiguity component. I guided a junior scientist on the ranking model — I gave the initial PEPNet direction with my own prototyping, then let them own implementation through regular design reviews. And I partnered with engineers on model serving and latency optimization. A key part of my role was alignment at the interfaces — making sure the generated child queries from the orchestration layer could actually be consumed by the retrieval system, which required several iterations of co-design with the retrieval team."

How to Control Pacing in the Interview

Strategy: Give the overview (~4 min), then let them drive depth.

If interviewer asks...	Go to...
"Tell me more about the LLM part"	Expand on prompt design, output schema, evaluation (see Q2, Q3 below)
"What were the trade-offs?"	V1 → V2 evolution story (already embedded, expand on distillation)
"How did you evaluate?"	Offline intent accuracy + generation quality + online A/B (see Q3)
"How did you lead the team?"	Leadership section + Q8 (cross-functional friction)
"What are the risks?"	Over-confident intent, generation failure, latency (see Q1, Q6)
"How does this apply here?"	Bridge to Apple (see Q5 below)
"Tell me about the ranking model"	PEPNet gating + multi-objective (already embedded, expand on weight tuning)

Key principle: Your initial narrative plants multiple hooks. The interviewer picks which thread to pull. You never over-explain upfront — you let them choose the depth.

Part A: Prepared Questions & Answers (From Your Original Prep, Polished)

Q1: "What were the trade-offs?" (Hooked from main narrative)

"The system went through two iterations driven by latency and system trade-offs.

In V1, we implemented a semi-real-time orchestration pipeline. We were experimenting with a 32B LLM to validate whether LLM-based personalization could effectively infer user shopping intent and generate useful child queries for downstream retrieval. Because the model was heavy, latency was the main constraint, so we moved inference to a T-1 session pipeline — the model analyzed the user's previous session behavior offline to infer a shopping mission and generate child queries. Results were cached using customer ID and product type, so when the user returned in the next session we could quickly retrieve the pre-generated queries.

However, through analysis we identified two key limitations. First, the cache key used product type instead of exact query — this was necessary to keep the key space manageable (from millions of possible queries down to ~2000 product types), but it caused granularity mismatches. For example, 'bedroom lamp' and 'living room lamp' mapped to the same product type, so the personalization couldn't distinguish between them. Second, the 32B model required about 23 seconds of processing time, which meant we occasionally missed recommendation opportunities if the user returned quickly.

These observations motivated V2. We distilled the 32B model into a task-specific 7B model, which significantly reduced inference latency to sub-100ms. We also introduced an intent detection gate: if the user's recent sequence showed a strong behavioral pattern, we triggered the personalization pipeline; otherwise we fell back to a lightweight discovery strategy. This hybrid design maintained personalization quality while meeting real-time latency requirements."

Q1.1: "What is the biggest risk of the system?" (Hooked)

"I think there are a few key risks.

The first is incorrect or over-confident intent inference. Since we're using LLMs to infer high-level intent from limited signals, there's always a chance the model makes a confident but wrong interpretation. If we over-personalize based on that, it hurts relevance and user trust. To mitigate this, we designed the confidence threshold mechanism — below 0.7 we don't personalize — and the downstream relevance gate filters out candidates that don't actually match the original query semantics.

The second risk is inconsistency in LLM outputs. Because the system relies on generated structured signals like queries, any format instability can break downstream retrieval or reduce coverage. So we enforce constrained decoding with structured output formats, monitor parse rates as part of evaluation, and

have fallback paths if generation fails.

The third risk is system latency and cost. Introducing an LLM into a real-time search serving path can significantly increase latency. We addressed this through model distillation, the intent detection gate that reduces unnecessary inference by 60%, and a strict timeout with graceful fallback.

Finally, there's an exploration-exploitation risk. If we over-optimize for short-term click signals, we may reduce diversity or hurt long-term marketplace health. So we monitor not just session metrics but 7-day return rate and purchase category breadth.

Overall, designing this system requires not just improving intent understanding, but carefully managing uncertainty, reliability, and system constraints."

Q2: "How did you lead the team?"

"This was a complex system redesign spanning intent modeling, retrieval, ranking, and infrastructure. One of the initial challenges was clearly defining the problem and translating it into a coherent system design that people with different backgrounds could execute on.

The team had different levels of experience across domains. Junior scientists were strong in modeling but less experienced in system-level thinking, while engineers were focused on scalability but not always aligned with modeling requirements and latency control. So a key part of my role was breaking down the problem and aligning everyone toward a unified design.

I started by defining the overall architecture and decomposing it into clear modules — the orchestration layer, retrieval integration, and ranking model — then assigned ownership based on each person's strengths. I remained hands-on for the orchestration layer since it was the core decision-making component with the highest ambiguity. I guided a junior scientist on the ranking model: I gave the initial PEPNet direction with my own prototyping, then let them own implementation through regular design reviews. And I partnered with engineers on model hosting and latency optimization.

Throughout the project, I focused heavily on cross-component alignment. For example, ensuring the generated child queries from the orchestration layer could be effectively consumed by the retrieval system required several iterations of co-design with the retrieval team. We had to jointly define interfaces, shared metrics, and fallback behaviors.

In terms of mentorship, I focused on helping junior scientists move beyond model-level thinking to system-level and product-level reasoning — how modeling decisions impact latency, user experience, and business metrics. This approach allowed us to execute efficiently while growing the team's capabilities."

Q3: "How do you evaluate your LLM / personalization system?"

"I think about evaluation in two layers: whether we correctly understand user intent, and whether that translates into better engagement and business

outcomes.

For offline evaluation, the main challenge is that the system is generative — it produces structured queries rather than directly ranking items. So we need to convert outputs into measurable signals.

First is intent prediction quality. We translated the generative task into a discriminative framework, which is important for RecSys because ultimately the outcome is binary — customer clicks or doesn't. We treat the generated child queries as a proxy for user intent and evaluate whether they can predict future user actions. Concretely, given historical behavior up to time T , we generate queries and check whether they align with what the user actually engages with at $T + 1$. We use both exact matching and semantic matching (embedding cosine > 0.7), since there are often multiple valid representations of the same intent. Our model achieves 72% semantic match rate vs 45% for a non-personalized baseline.

Second is generation consistency and usability. Since the system feeds into downstream retrieval pipelines, outputs must follow a structured JSON format. We evaluate parse success rate — targeting above 98%. If generation is inconsistent, even semantically correct outputs can't be consumed by the system, which directly impacts recommendation coverage.

Online, the key question is whether better intent understanding leads to better user experience and business outcomes. We track CTR, downstream CVR, and coverage (what percentage of sourced candidates came from the personalized path vs fallback). We evaluate end-to-end — not just model-level improvements but overall system impact, since intent understanding feeds into retrieval and ranking.

We also have guardrail metrics: if relevance scores drop or coverage declines, we automatically reduce traffic allocation to the treatment group."

Q4: "Why did you use LLM instead of other models?"

"I've worked with both traditional methods and LLM-based approaches, so I've seen the trade-offs clearly.

With traditional methods like intent classification or sequential recommendation models, the core limitation is that they rely on a fixed or implicit representation of user intent. In practice, user intent is highly dynamic and often under-specified, so these models either restrict us to a predefined label space or learn implicit patterns that are difficult to interpret and control. For example, with running shoes alone there are hundreds of possible intents — cushioned vs stability, road vs trail, racing vs training, specific brand affinity. A fixed taxonomy can't capture this compositional, evolving space.

Another limitation is that these approaches are inherently restrictive — they predict from a predefined space — which makes it difficult to capture compositional or higher-level intent, especially when we want to incorporate multiple signals like session behavior, context, and real-time interactions.

What LLMs enable is a generative approach. Instead of predicting from a fixed

space, the model reasons over user behavior and generates structured representations of intent that aren't limited to predefined categories. In our system, this is particularly useful because we translate user behavior into actionable signals — structured queries — that directly guide downstream retrieval.

More importantly, LLMs allow us to integrate richer and more heterogeneous signals — including longer user sequences and contextual information — and leverage world knowledge to make more robust inferences, especially when signals are sparse or ambiguous.

In terms of model sizing: we tested 3B, 7B, and 14B variants. 3B lost too much generation quality — it produced overly generic queries. 14B didn't improve meaningfully but doubled latency. 7B achieved 95% of the 32B teacher's accuracy at one-quarter the latency — that was our sweet spot."

Part B: Additional Predicted Questions (ML HM Depth Probes)

Q5: "How do you handle cases where the LLM generates bad/irrelevant child queries?"

"This was one of our biggest operational challenges. We addressed it at three levels.

First, constrained generation: we use structured output formats with JSON schema enforcement. The model must produce queries within a predefined product taxonomy, which bounds the output space and prevents hallucinated categories.

Second, a downstream relevance gate: even if a child query is generated, the retrieval results still go through our relevance model before ranking. If the retrieved candidates don't meet a minimum relevance threshold against the original user query, they're filtered out.

Third, monitoring and automated safeguards: we track parse success rate (targeting > 98%), semantic similarity between generated queries and actual next-session user behavior, and online guardrail metrics. If a treatment group shows declining relevance scores, we automatically reduce traffic allocation.

In practice, about 5-8% of generated queries are 'drifted' — semantically valid but not matching what the user actually does next. The relevance gate catches most of these before they impact the user experience."

Q6: "What does the prompt look like? How did you design it?"

"The prompt has three sections: system instructions defining the task and output format, the user's behavioral context formatted as a structured sequence, and the generation instruction.

The system instruction specifies: 'You are a shopping intent analyzer. Given a user's recent browsing and purchase history, infer their current shopping

mission and generate 3-5 specific search queries that would help them find relevant products.'

The context section includes the last 20 interactions with timestamps, action types (view/click/cart/purchase), product titles, and categories — formatted as a structured list.

The generation instruction asks for JSON output with `intent_summary`, `confidence` (0-1), and `child_queries` array.

We iterated on prompt design extensively. The key insight was that including action types — not just product names — dramatically improved intent inference. A user who VIEWED 5 running shoes has different intent than one who PURCHASED one and is now viewing socks.

We also found that providing 2-3 few-shot examples in the prompt improved output format consistency from 89% to 98% parse rate."

Q7: "How does this apply to Apple's App Store ads matching?"

"The core architecture maps directly. In the App Store, a user searches 'photo editor' — that's the mid-funnel query. As they browse — click on Lightroom's page, read Photoshop's description, skip the casual filter apps — we're observing session-level intent signals.

The LLM orchestration layer would infer: 'this user wants professional color grading tools, not casual filters.' It generates refined queries like 'professional photo color correction' and routes to semantic retrieval and app-to-app co-install retrieval using Lightroom as a seed.

The key difference at Apple: all of this must be session-scoped and privacy-compliant. No persistent user profiles, no cross-app tracking. But the architecture I built is actually well-suited for this — it's designed to work within a single session's signals, making real-time inferences without needing long-term user history. The session ends, the state is discarded.

For the ranking component, the placement-aware architecture maps to different ad surfaces — App Store search, App Store browse tabs, Apple News, Apple Maps. Each has different user behavior patterns and objectives, exactly the problem PEPNet-style gating solves."

Q8: "What was the hardest technical challenge?"

"The hardest challenge was managing the latency-quality trade-off in the LLM layer while maintaining system reliability.

The LLM inference sits in the critical path of ad serving. If it's slow or fails, we block the entire recommendation response. So I designed three safeguards:

First, the intent detection gate: a lightweight binary classifier (logistic regression on session features, < 1ms) decides whether to invoke the LLM at all. This reduces unnecessary calls by 60%.

Second, timeout with graceful fallback: if LLM inference exceeds 80ms, we

abort and fall back to the discovery strategy. The user never sees a timeout — they just get non-personalized results.

Third, async pre-computation for repeat visitors: if a user has been on the page for a while, we speculatively pre-compute their intent in the background so subsequent interactions can use cached results.

The combined system achieves p50 latency of 45ms and p99 of 92ms for the personalized path, while maintaining 99.7% availability through the fallback mechanism."

Q9: "How do you handle the exploration/exploitation trade-off?"

"If we always personalize based on inferred intent, we risk filter bubbles — showing users only what we think they want, missing serendipitous discoveries.

We balance this at two levels. First, the confidence threshold: when intent confidence is below 0.7, we explicitly route to the discovery path which expands into related categories. This ensures roughly 30% of traffic gets exploration-oriented recommendations.

Second, within the personalized path: the child queries include both 'core intent' queries and 'adjacent exploration' queries one step away from the core intent. So even personalized results have built-in diversity.

We monitor long-term metrics — not just session CTR but 7-day return rate and category breadth of purchases — to ensure we're not over-narrowing the user's experience."

Q10: "Tell me about a disagreement or challenge with cross-functional partners."

"The main friction was with the retrieval team. Our LLM generates structured child queries, but their system was designed for user-typed search queries — not system-generated ones. Our generated queries were sometimes more specific or used terminology that didn't exist in the product index.

For example, we'd generate 'cushioned neutral road running shoes for daily training' but the retrieval index was optimized for shorter queries like 'running shoes men.' Our longer, more specific queries actually returned fewer results — hurting coverage.

I worked with the retrieval team on three solutions: first, adding semantic matching alongside BM25 so our queries could match on meaning rather than exact terms. Second, implementing a query truncation strategy that progressively shortens queries if initial retrieval returns below a coverage threshold. Third, jointly defining a shared metric — relevant candidates per query — that aligned both teams.

This took about 3 weeks of back-and-forth, but resulted in a much more robust integration. The lesson: when you introduce a generative component upstream, you need to co-design the interface with downstream consumers."

Questions YOU Should Ask the Hiring Manager

1. "How does the team currently handle intent understanding for ad matching — is it primarily keyword-based, or is there already a semantic layer?"
 2. "What's the team's philosophy on the latency-quality trade-off for real-time personalization?"
 3. "How do you balance advertiser objectives (ROAS) with user experience (relevance) when they conflict?"
 4. "What does the experimentation infrastructure look like — how quickly can you get a model change into A/B testing?"
 5. "Given Apple's privacy constraints, how does the team approach personalization without persistent user profiles?"
-

Key Talking Points to Hit (Checklist)

For an ML Hiring Manager, make sure you cover:

- ☐ **Problem framing:** Why was this important? Business impact of the gap.
- ☐ **Technical architecture:** Two clear components, each with specific model choices and WHY.
- ☐ **Iteration story:** V1 (async 32B) → V2 (real-time 7B) shows you learn and adapt.
- ☐ **Quantitative results:** \$40M revenue, 1.5% CTR, 10% RoAS, 0.11% CVR, 0.18% click.
- ☐ **Leadership:** Decomposed problem, assigned ownership, mentored junior, aligned cross-functional.
- ☐ **Risk management:** Identified and mitigated over-confident intent, generation failures, latency.
- ☐ **Evaluation rigor:** Offline (intent accuracy, parse rate) + Online (A/B with guardrails).
- ☐ **Bridge to Apple:** Map your system to App Store ads + privacy-compliant session-level personalization.